

Google & GitHub Dorking

Google Dorking (also called Google Hacking) is the practice of using advanced search operators to find specific information indexed by Google that isn't easily surfaced through normal searches — exposed files, login portals, misconfigured servers, and sometimes sensitive data accidentally made public. GitHub Dorking applies the same mindset to GitHub's code search, and is arguably even more valuable in modern recon, since developers accidentally leak far more sensitive data on GitHub than on indexed websites.

Both are core **passive reconnaissance** techniques — mapping directly to the Reconnaissance stage of the Cyber Kill Chain and to MITRE ATT&CK's Reconnaissance tactic (**TA0043**), specifically **T1596** (Search Open Technical Databases) and **T1593.003** (Search Open Websites/Domains: Code Repositories).

1. Why This Matters

Search engines crawl and index publicly accessible content — including things website owners never intended to be discoverable, like backup files, admin panels, or exposed configuration files. GitHub, similarly, hosts hundreds of millions of public repositories where developers routinely (accidentally) commit API keys, credentials, `.env` files, private keys, and internal documentation.

Since both techniques are **passive** — you're not touching the target's live infrastructure, just querying a third-party index — they're among the lowest-risk, highest-value early recon steps in any authorized pentest or bug bounty engagement.

Key distinction:

- **Google Dorking** finds what's *indexed and public-facing* (pages, files served by web servers)
 - **GitHub Dorking** finds what developers *directly wrote into code*, often without realizing it's permanently searchable and version-tracked forever — even old commits "removed" later still exist in git history
-

PART 1: GOOGLE DORKING

2. Core Search Operators

Operator	Function	Example
<code>site:</code>	Restrict results to a specific domain	<code>site:example.com</code>
<code>inurl:</code>	Find a term in the URL	<code>inurl:admin</code>
<code>intitle:</code>	Find a term in the page title	<code>intitle:"index of"</code>
<code>intext:</code>	Find a term in the page body	<code>intext:"password"</code>
<code>filetype:</code>	Restrict to a specific file extension	<code>filetype:pdf</code>
<code>ext:</code>	Same as filetype (alias)	<code>ext:sql</code>
<code>cache:</code>	View Google's cached version of a page	<code>cache:example.com</code>
<code>link:</code>	Find pages linking to a URL (largely deprecated now)	<code>link:example.com</code>
<code>related:</code>	Find sites related to a given domain	<code>related:example.com</code>
<code>-</code>	Exclude a term	<code>site:example.com -inurl:www</code>
<code>"..."</code>	Exact phrase match	<code>"confidential report"</code>
<code>*</code>	Wildcard	<code>admin * login</code>
<code>OR / </code>	Either term	<code>filetype:pdf OR filetype:doc</code>
<code>AROUND(n)</code>	Terms within n words of each other	<code>security AROUND(3) breach</code>

3. Practical Dorks for Reconnaissance

a) Finding Exposed Directory Listings

```
intitle:"index of" "parent directory"
intitle:"index of/" backup
```

Open directory listings often expose backups, logs, or source code that was never meant to be browsable.

b) Finding Login Portals (mapping attack surface)

```
site:example.com inurl:login
site:example.com intitle:"login"
inurl:admin intext:"username" intext:"password"
```

c) Finding Exposed Configuration Files

```
filetype:env "DB_PASSWORD"  
filetype:xml inurl:"web.config"  
filetype:ini "wp-config"  
inurl:wp-config.php intext:"DB_PASSWORD"
```

`.env` and `wp-config.php` exposure is extremely common in real-world bug bounty findings — a misconfigured server sometimes serves these files as plain text instead of executing them.

d) Finding Exposed Database Files / Dumps

```
filetype:sql intext:"INSERT INTO" intext:"password"  
filetype:sql "-- phpMyAdmin SQL Dump"
```

e) Finding Exposed Log Files

```
filetype:log inurl:"error_log"  
filetype:log intext:"password"
```

f) Finding Exposed Credentials / API Keys in Public Repos or Pastebins

```
site:pastebin.com intext:"api_key"  
site:github.com "api_key" "example.com"
```

This is exactly why secret-scanning tools (TruffleHog, GitLeaks) exist for defenders — accidental credential commits are a huge real-world breach vector.

g) Finding Vulnerable/Outdated Software Versions

```
intext:"Powered by phpBB 2.0" inurl:forum  
intitle:"Apache Tomcat" intitle:"Error Report"  
intext:"Welcome to nginx" intitle:"Welcome to nginx!"
```

Useful for identifying default/unconfigured installs that were never hardened after deployment.

h) Finding Exposed Webcams / IoT Devices (classic dork category)

```
intitle:"webcamXP 5"  
inurl:"view/index.shtml"
```

i) Finding SQL Error Messages (indicates possible injection points)

```
intext:"sql syntax near" intext:"error"  
intext:"Warning: mysql_connect()"
```

j) Finding Exposed Cloud Storage

```
site:s3.amazonaws.com "example"  
intitle:"index of" "bucket"  
inurl:blob.core.windows.net
```

Misconfigured S3 buckets are one of the most commonly reported cloud misconfigurations in bug bounty programs — this dork is a fast passive check before running dedicated tools like `s3scanner`.

k) Finding Exposed Git Repositories (on a live web server, not GitHub.com)

```
intitle:"index of" ".git"  
inurl:"/.git/config" filetype:config
```

An exposed `.git` folder can let an attacker reconstruct the entire source code history using tools like `git-dumper`:

```
git-dumper http://target.com/.git/ ./dumped-repo
```

4. Domain-Scoped Recon Combo (How It's Actually Used in Practice)

Real recon usually chains a target domain with multiple dork categories:

```
site:example.com filetype:pdf  
site:example.com filetype:xls OR filetype:xlsx  
site:example.com inurl:login  
site:example.com intext:"internal use only"
```

```
site:example.com -www inurl:dev
site:example.com ext:log
```

The `-www inurl:dev` combo is worth calling out: it's hunting for **subdomains** (like `dev.example.com`, `staging.example.com`) that Google has indexed but that aren't linked from the main site, since dev/staging environments are frequently less hardened than production.

5. Google Hacking Database (GHDB)

The **Exploit-DB Google Hacking Database** (originally started by Johnny Long) is a massive, categorized, community-maintained collection of dorks, organized by category:

- Footholds
- Files containing usernames
- Sensitive directories
- Web server detection
- Vulnerable files
- Error messages
- Files containing juicy info (passwords, etc.)
- Pages containing login portals
- Advisories and vulnerabilities

Worth browsing directly — it's continuously updated and organized exactly the way you'd want for structured study (search: "Google Hacking Database exploit-db").

6. Tools That Automate Google Dorking

```
# theHarvester - OSINT tool, can use search engines for rec
on
theHarvester -d example.com -b google

# Pagodo - automates GHDB dork queries against a target dom
ain
python3 pagodo.py -d example.com -g dorks.txt
```

```
# dorks-eye, GooDork - dedicated dork automation tools
```

These tools take a target domain and run it against hundreds of GHDB-style dorks automatically — far more practical than manual querying once you're past the "learning the operators" stage.

PART 2: GITHUB DORKING

7. Why GitHub Dorking Is Its Own Category

GitHub hosts hundreds of millions of public repositories, and developers routinely (accidentally) commit:

- API keys, tokens, and secrets
- Database credentials and connection strings
- Internal hostnames, IP addresses, VPN configs
- `.env` files, private keys, AWS credentials
- Internal documentation, source code for proprietary tools
- Old commits containing secrets that were "removed" later but still exist in git history

8. GitHub Search Basics

```
org:company-name  
user:username  
repo:owner/repo-name  
filename:config.php  
extension:env  
language:python  
path:config
```

Combine with keywords:

```
org:company-name password  
org:company-name filename:.env  
org:company-name extension:pem
```

Note: GitHub's code search syntax/coverage has changed over the years and sometimes requires being logged in — worth checking GitHub's current search docs, since older dork lists occasionally reference syntax that no longer works exactly the same way.

9. Practical GitHub Dorks

a) Finding Exposed API Keys / Secrets

```
filename:.env DB_PASSWORD
extension:json api_key
extension:yaml aws_access_key_id
"api_key" "secret" extension:py
```

b) Finding AWS Credentials

```
extension:json "aws_access_key_id"
filename:credentials aws_secret_access_key
"AKIA" extension:env
```

AKIA is the standard prefix for AWS access key IDs — one of the highest-signal GitHub dorks that exists, since that prefix rarely appears by coincidence.

c) Finding Database Credentials / Connection Strings

```
extension:yaml "mysql://"
extension:json "mongodb+srv://"
filename:database.yaml password
"DB_PASSWORD" extension:env
```

d) Finding Private Keys

```
filename:id_rsa
extension:pem private
"BEGIN RSA PRIVATE KEY"
"BEGIN OPENSSH PRIVATE KEY"
```

e) Finding Internal/Config Files Scoped to an Organization

```
org:company-name filename:config
org:company-name extension:env
org:company-name "internal" path:docs
```

f) Finding Slack/Discord Webhooks

```
"hooks.slack.com/services"
"discord.com/api/webhooks"
```

g) Finding Google API Keys

```
"AIza" extension:js
```

AIza is the standard prefix for Google API keys — same logic as the AWS **AKIA** dork.

h) Finding Hardcoded Passwords in Source Code

```
extension:py "password ="
extension:js "password:"
extension:php "$password ="
```

10. Curated GitHub Dork Lists

Similar to the GHDB, there are community-maintained GitHub dork lists worth knowing — searchable as "github dorks list," typically organized by category: cloud credentials, database strings, private keys, tokens by service (Slack, Stripe, Twilio, etc.).

11. Automated GitHub Secret-Scanning Tools

```
# GitLeaks - scans a repo's entire commit history
gitleaks detect --source=/path/to/repo
gitleaks detect --source=/path/to/repo --report-path=report.json

# TruffleHog - strong at high-entropy string detection across commit history
trufflehog git https://github.com/owner/repo
```



```
trufflehog github --org=company-name

# GitHound - combines dork-style searching with automated aggregation
githound --dig -q "company-name"

# Shhgit - watches GitHub's public commit firehose live, flags secrets in real time
```

GitLeaks and TruffleHog matter because a secret removed in a later commit still exists in git history unless the history itself was rewritten and force-pushed.

12. GitHub Recon Beyond Just Secrets

```
org:company-name # all public repos owned by the org
org:company-name in:readme "internal" # repos referencing internal tools/processes
org:company-name language:dockerfile # infrastructure/deployment configs
```

Employee-level recon: contributor usernames can be cross-referenced with LinkedIn to build an employee list. Commit history sometimes reveals real names/emails even when GitHub profiles use pseudonyms:

```
git log --all --format='%ae' | sort -u
```

Developers frequently forget their global git config uses their real corporate email, even on personal side projects — silently leaking it into every commit.

13. Combining Google + GitHub Dorking

```
# Find company GitHub orgs/repos indexed by Google
site:github.com "company-name"
site:github.com "company-name" "internal"

# Find leaked credentials referencing GitHub on paste sites
site:pastebin.com "github.com/company-name"
```

```
# Find exposed .git folders on live web servers (not GitHub.com itself)
intitle:"index of" ".git"
```

14. Why This Category Is So High-Value

Accidental credential leaks on GitHub are consistently one of the most common **initial access vectors** in real breaches, because:

- Developers often work fast and commit `.env` files by mistake
- Removing a file in a later commit doesn't remove it from history
- Public repos get scraped continuously by both researchers and attackers within minutes of going public
- Automated bots actively monitor GitHub's public commit stream in real time (as Shhgit demonstrates defensively)

GitHub Dorking is one of the few recon techniques that can skip straight from Reconnaissance to a *working credential* — collapsing several kill-chain stages if a live secret is found, which is why it's treated as a high-priority check in modern engagements.

15. Defending Your Own GitHub Repos (Directly Actionable for You)

a) Never commit secrets — use environment variables + `.gitignore`

```
# .gitignore
.env
*.pem
config/secrets.yml
```

b) If a secret was already committed, deleting the file isn't enough — purge history

```
bfg --delete-files secrets.env
git reflog expire --expire=now --all && git gc --prune=now
--aggressive
```

c) **Enable GitHub's built-in secret scanning + push protection** (Settings → Security → Code security and analysis)

d) **Rotate any credential immediately if it's ever exposed**, even briefly.

e) **Pre-commit hooks to catch secrets before they're ever pushed:**

```
gitleaks protect --staged
```

Mentioning "implemented GitLeaks pre-commit scanning across my repos" is a small but real credibility signal in interviews and on your GitHub README.

16. Legal & Ethical Boundaries (Applies to Both)

- **Searching Google/GitHub itself is not illegal** — you're querying a public search index, not attacking a system
- **However, accessing data you find** (like a `.sql` dump with real customer PII, using a leaked AWS key, or logging into an exposed admin panel with found creds) **crosses into unauthorized access** — this is where laws like the **CFAA (US)** or **IT Act 2000 (India)** apply
- For professional work, dorking should only be performed as part of an **authorized engagement** with signed scope/rules of engagement
- Even in bug bounty programs, most programs explicitly define what's in-scope for "passive recon" — check program rules before running domain-wide dork sweeps, since some consider dorking sensitive if it touches out-of-scope subdomains
- Using harvested/leaked credentials for real, non-consensual access is straightforwardly illegal — the value of this skill is entirely in **authorized** assessments and CTF/lab contexts

17. Defensive Angle (Relevant to Your SOC Interest)

a) `robots.txt` (weak control — doesn't prevent indexing of linked pages, just crawler behavior)

```
User-agent: *  
Disallow: /admin/  
Disallow: /backup/
```

b) **noindex meta tag** (stronger — actually tells search engines not to index)

```
<meta name="robots" content="noindex, nofollow">
```

c) **Google Search Console — "Remove outdated content" tool** — lets domain owners actively request removal of already-indexed sensitive URLs.

d) **Proper server configuration**

- Disable directory listing (`Options -Indexes` in Apache)
- Never serve `.env`, `.git`, `.sql`, or backup files from a web-accessible directory in the first place — the real fix is architectural, not just hiding it from crawlers
- Regular external attack-surface scans using the same dorks attackers would use — many organizations run scheduled GHDB sweeps against their own domains as part of continuous attack surface management

e) **DLP and periodic self-auditing** — running your own `site:yourcompany.com filetype:pdf` sweep periodically is a legitimate, low-effort attack surface monitoring practice worth recommending in a security assessment.

f) **GitHub-specific defenses** — native secret scanning + push protection (see Section 15), plus organization-wide policies restricting what can be committed to public repos.

18. Kill Chain / ATT&CK Mapping

Framework	Google Dorking	GitHub Dorking
Cyber Kill Chain	Reconnaissance	Reconnaissance → can directly enable Initial Access
MITRE ATT&CK	T1596 (Search Open Technical Databases)	T1593.003 (Search Open Websites/Domains: Code Repositories), T1552.001 (Unsecured Credentials: Credentials In Files)

Quick Reference Cheat Sheet

Google Dorking

site:example.com	restrict to domain
inurl:admin	term in URL
intitle:"index of"	open directory listing
filetype:pdf / ext:sql	restrict file type
intext:"password"	term in page body
"exact phrase"	exact match
-term	exclude term
site:example.com filetype:env	exposed env files
site:example.com inurl:login	login portals
intitle:"index of" ".git"	exposed git repo
s (on a live server)	
site:pastebin.com "api_key"	leaked credentials

GitHub Dorking

```
org:company-name filename:.env
org:company-name extension:pem
"AKIA" extension:env           # AWS key prefix
"AIZA" extension:js           # Google API key prefix
"BEGIN RSA PRIVATE KEY"
filename:id_rsa
site:github.com "company-name" "internal"
```

Automation / Scanning Tools

```
theHarvester -d example.com -b google
python3 pagodo.py -d example.com -g dorks.txt

gitleaks detect --source=/path/to/repo
trufflehog git https://github.com/owner/repo
```

```
git-dumper http://target.com/.git/ ./dumped-repo  
git log --all --format='%ae' | sort -u
```
